REPORTE DE ACTIVIDAD

Universidad Autónoma de Nuevo León

Facultad de Ingeniería Mecánica y Eléctrica

<table>
<tr><td colspan="2">Equipo 5. Integrantes:

• Manuel Alexander Fuentes Contreras
• Rodolfo Antonio Ramírez Garza</td><td>Matrículas:

• 2134970
• 2119176</td></tr>
<tr><td colspan="2">Nombre del curso: Componentes de lógica de negocio</td><td>Nombre del profesor: Miguel Ángel Gómez Marroquín</td></tr>
<tr><td colspan="2">Modulo Netacad:</td><td>Actividad: Reporte de Actividad U3</td></tr>
<tr><td colspan="3">Fecha: 14/10/2026</td></tr>
<tr><td colspan="3">Bibliografia (Formato APA)
Evans, E. (2003). Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley.

Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.

Larman, C. (2004). Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development (3rd ed.). Prentice Hall.</td></tr>
</table>

Objetivo:

Diseñar y fundamentar la propuesta de arquitectura de software para la empresa **TecnoStock**, transformando los productos de análisis de negocio de la Unidad 2 (BRD, historias de usuario con criterios Gherkin, glosario de términos y diagramas de procesos BPMN) en un modelo computacional robusto. La propuesta busca estructurar la lógica del sistema dividiendo las responsabilidades en capas claras (*Domain*, *Application*, *Infrastructure*), protegiendo las invariantes críticas del dominio —tales como la unicidad de los números de serie en electrónicos, la prevención de dobles ventas en apartados y el control de devoluciones— mediante la integración de patrones de arquitectura empresarial, diseño guiado por el dominio (DDD) y la formulación de registros de decisiones de arquitectura (ADR).

Enlace al Repositorio de GitHub:

- **Repositorio oficial de la Unidad 3:** https://github.com/AlexMist07/TecnoStock-Arquitectura

Procedimiento y Desarrollo de la Arquitectura

1. Antecedentes del Caso TecnoStock y Selección del Patrón de Lógica de Negocio

Contexto y Problemática del Negocio

La empresa TecnoStock opera una cadena de 5 sucursales dedicadas a la comercialización de productos electrónicos y accesorios de tecnología. El análisis previo de la Unidad 2 identificó serias inconsistencias operativas en el punto de venta (PdV) Windows actual y en la comunicación con el supervisor remoto:

1. **Desfases de inventario e inconsistencia en números de serie:** Se presentaban dificultades para rastrear el historial de dispositivos individuales (laptops, celulares, tablets) que requieren número de serie obligatorio.
2. **Dobles ventas de productos apartados:** Los artículos reservados por un cliente con anticipo eran vendidos por error a otros clientes debido a la falta de una fórmula unificada y transaccional de stock disponible.
3. **Criterios de devolución disparidad entre sucursales:** El personal de caja aplicaba políticas de manera arbitraria al no existir un motor centralizado que validara los límites de tiempo (30 días), la existencia de comprobantes de pago o los montos que requieren autorización del supervisor.

Preparación del Caso y Selección de Casos de Uso

Para delimitar el alcance técnico del diseño para la empresa TecnoStock, se seleccionaron cuatro casos de uso principales extraídos de las historias de usuario y de los flujos BPMN, abarcando flujos normales y escenarios con excepciones:

ID	Caso de uso	Actor	Reglas relacionadas	Resultado esperado
CU-01	Consultar disponibilidad (US01)	Vendedor	Consultar stock en las 5 sucursales separando disponible y apartado	Devuelve existencias reales sin bloquear stock
CU-02	Registrar venta con serial (US02, US03)	Vendedor / Cajero	Obligatoriedad de serial en electrónicos; unicidad global; pago completo	Venta registrada, serial bloqueado, comprobante emitido
CU-03	Crear y completar apartado (US04, US06)	Vendedor / Cliente	Anticipo mínimo del 25%; vencimiento fijo a 14 días; bloqueo de stock	Stock reservado; conversión a venta formal al liquidar
CU-04	Solicitar y resolver devolución (US08, US09)	Cajero / Supervisor	Plazo de 30 días; comprobante O aprobación de supervisor si > $500	Reembolso por método original o cambio con diferencia

Tabla 1. Casos de uso seleccionados para el diseño de TecnoStock.

Evaluación Comparativa de Patrones

Para solucionar la problemática, se analizaron los cuatro patrones fundamentales de lógica de negocio propuestos por Martin Fowler en *Patterns of Enterprise Application Architecture*:

- **Transaction Script:** Agrupa la lógica de negocio en procedimientos lineales por cada transacción de la base de datos. Si bien es fácil de implementar en sistemas pequeños, en TecnoStock conduciría a la duplicación masiva de código y a condicionales dispersos para validar las 12 reglas de negocio del caso.
- **Table Module:** Organiza la lógica alrededor de tablas de la base de datos. Se descartó por acoplar fuertemente la aplicación a la estructura relacional y dificultar la representación de comportamientos ricos y máquinas de estado complejas.
- **Active Record:** Fusiona la lógica del negocio con el acceso a datos en los mismos objetos. Resulta insuficiente cuando las entidades tienen ciclos de vida estructurados o requieren validaciones cruzadas entre múltiples conceptos antes de persistir los cambios.
- **Domain Model:** Estructura la lógica mediante un red conceptual de objetos de dominio (entidades, objetos de valor y agregados) que encapsulan datos y comportamiento de forma pura, desacoplada de la infraestructura y la base de datos.

Justificación Arquitectónica Detallada: Selección del Patrón de Lógica de Negocio

Empresa Asignada: TecnoStock (Retail de Electrónica y Componentes)

1. Planteamiento de la Decisión y Contexto Operativo

Para el desarrollo del núcleo transaccional del sistema de TecnoStock —una cadena de cinco sucursales dedicada a la comercialización de computadoras, celulares, accesorios y componentes—, el equipo de arquitectura ha seleccionado la combinación del patrón Domain Model (*Modelo de Dominio*) de Martin Fowler, coordinado a través de una Service Layer (*Capa de Servicios*).

Asimismo, se establece una estrategia pragmática y acotada de lectura: para aquellas operaciones puramente analíticas o de consulta que no alteran el estado ni aplican reglas de negocio (tales como la consulta de disponibilidad en tiempo real o el reporte general de stock por sucursal correspondientes a las historias US01, US15 y US17), se permiten accesos directos de lectura optimizada bajo un enfoque tipo *Transaction Script / Read Model*, evitando sobrecargar el modelo de dominio con consultas pesadas que no protegen invariantes.

La elección de *Domain Model* no responde a una tendencia tecnológica ni a criterios genéricos de "modernidad", sino a la necesidad imperativa de resolver los fallos estructurales y operativos detectados en la evaluación del estado actual (*AS-IS*) de la compañía.

2. Análisis de los Problemas Concretos del Caso que Justifican el Patrón

El análisis de requerimientos de negocio (BRD) de TecnoStock evidenció que los desafíos prioritarios (necesidades N1, N2, N3, N5 y N7) no consisten en una simple captura, almacenamiento o despliegue tabular de datos, sino en la vulnerabilidad de las invariantes de negocio y políticas comerciales complejas. En la operación actual, la ausencia de un núcleo de dominio centralizado provoca consecuencias financieras directas que el nuevo software debe erradicar por completo:

- Venta accidental de productos apartados (Invariante N2): Actualmente, un producto reservado por un cliente mediante un anticipo mínimo del 25% puede ser vendido por error a otro comprador en el punto de venta debido a que la fórmula de disponibilidad no se evalúa de manera transaccional. *Domain Model* resuelve este problema al concentrar la regla dentro del agregado StockSucursal, cuya fórmula exclusiva (disponible = total − reservado − dañado − en tránsito) rechaza de forma automática cualquier intento de venta si la mercancía se encuentra bloqueada bajo el estado RESERVADO.
- Dificultad en el control y unicidad de números de serie (Invariante N1): En la venta de equipos electrónicos de alto valor (laptops, smartphones, tablets), el sistema actual carece de un mecanismo estricto que garantice la unicidad global del número de serie. Con *Domain Model*, cada dispositivo se modela como un agregado independiente (NumeroDeSerie) que administra su propio ciclo de vida e historial inmutable, impidiendo que un mismo serial sea asignado a dos transacciones simultáneas.
- Inconsistencia en las políticas de devolución (Invariante N5): El personal de caja aplica criterios subjetivos y dispares entre las cinco sucursales al procesar devoluciones. El modelo propuesto centraliza la validación de plazos (30 días), la obligatoriedad del comprobante o la necesidad de autorización del supervisor en un *Domain Service* y objetos de especificación, garantizando un comportamiento totalmente estandarizado y auditable en toda la cadena.

3. Por qué los Demás Patrones de Fowler Resultan Insuficientes

Para fundamentar con rigor la decisión, se contrastó el comportamiento de TecnoStock frente a las alternativas tradicionales del catálogo de patrones de arquitectura empresarial (*PoEAA*):

1. Transaction Script (Descartado): Organiza la lógica mediante procedimientos lineales o scripts por cada transacción. Aunque su costo inicial es bajo, en TecnoStock propiciaría la dispersión de condicionales if/else a lo largo de las pantallas del PdV Windows y scripts de base de datos. Las 12 reglas implícitas del negocio terminarían duplicándose, lo que generaría discrepancias operativas inevitables al modificar una política comercial.
2. Table Module (Descartado): Agrupa la lógica alrededor de una sola clase por tabla de base de datos para manejar conjuntos de registros. Este patrón resulta inoperante al modelar comportamientos ricos y máquinas de estado complejas, tales como la transición de un Apartado (Activo, Completado, Cancelado, Vencido) o el ciclo de vida de un componente en inventario, acoplando indeseablemente la aplicación al esquema relacional de las tablas.
3. Active Record (Descartado): Fusiona los datos de la fila con la lógica de negocio básica en una misma clase. Si bien es útil para operaciones CRUD sencillas, colapsa cuando las entidades deben proteger invariantes cruzadas entre múltiples conceptos antes de persistir los cambios, filtrando detalles de infraestructura hacia la lógica de la aplicación.

4. Costos, Consecuencias y Contrapartidas (*Trade-offs*) Introducidas

La adopción de *Domain Model + Service Layer* no está exenta de costos técnicos; el equipo de arquitectura asume consciente y explícitamente las siguientes contrapartidas:

- Curva de aprendizaje elevada y mayor complejidad inicial: El desarrollo exige un esfuerzo de abstracción riguroso para modelar los límites de consistencia (*Bounded Contexts*), agregados y objetos de valor antes de escribir código operativo, lo que incrementa el tiempo de diseño previo.
- Incremento de clases e indirección: Se introduce una separación estricta mediante el patrón *Repository* y mapeadores de objetos, lo que eleva

notablemente el volumen inicial de clases en comparación con soluciones procedurales.

- Gestión de concurrencia: Al operar simultáneamente en cinco sucursales y canales remotos, el modelo requiere implementar un control de concurrencia optimista (con números de versión) en los agregados de stock para evitar bloqueos pesimistas lentos, obligando a la *Service Layer* a gestionar reintentos controlados en caso de colisiones de inventario.

5. Conclusión y Formalización

En conclusión, la elección de Domain Model + Service Layer para TecnoStock se encuentra plenamente justificada porque es el único enfoque arquitectónico capaz de proteger las invariantes críticas del negocio, unificar los criterios de operación de las sucursales y permitir una integración limpia y no intrusiva con el punto de venta Windows existente a través de adaptadores en la capa de servicios. Esta decisión queda formalmente registrada en el documento de arquitectura bajo el identificador ADR-001.

Figura 1. Tabla comparativa de selección del patrón de lógica de negocio para TecnoStock.

1.1 Tabla comparativa (patrones de Fowler, PoEAA)

Escala: ● cumple bien · ◐ cumple parcialmente · ○ cumple mal.

Criterio (origen en TecnoStock)	Transaction Script	Table Module	Active Record	**Domain Model**
Cantidad y complejidad de reglas (12 reglas implícitas, estados Activo/Completado/Cancelado/Vencido)	○ if/else dispersos	◐	◐	● reglas dentro de los objetos
Invariantes críticas: serial único (N1), cero dobles ventas (N2), anticipo ≥ 25%, vencimiento = creación + 14 días	○ se repiten en cada script	◐	◐	● protegidas por Aggregates
Política de devolución (30 días, comprobante O supervisor, > $500, daño) y su evolución	○ crece sin control	◐	◐	● Domain Service / Specification
Ciclos de vida con estados (Apartado, NumeroDeSerie, Devolución)	○	○	◐	● máquinas de estado en la entidad
Trazabilidad y auditoría de todo cambio (regla 9)	◐	◐	◐	● Domain Events

Criterio (origen en TecnoStock)	Transaction Script	Table Module	Active Record	**Domain Model**
Testabilidad sin base de datos (criterios Gherkin verificables)	◐	○	○ acoplado a BD	● dominio puro
Integración con el PdV Windows actual (restricción BRD)	● simple	◐	◐	● vía Service Layer (adaptador)
Costo inicial / curva de aprendizaje	● bajo	● bajo	● bajo	◐ mayor
Reportes simples (US17 stock por sucursal)	● ideal	●	●	◐ innecesario

2. Diseño Guiado por el Dominio (DDD) y Contextos Delimitados (*Bounded Contexts*)

Lenguaje Ubicuo

Uno de los pilares del diseño fue la alineación del vocabulario del software con los términos reales utilizados por los cajeros, vendedores, almaceneros y supervisores de TecnoStock. Se estableció un **Lenguaje Ubicuo** formal donde cada concepto del glosario de la Unidad 2 se mapeó a una construcción específica del modelo orientado a objetos, garantizando que los mismos términos se utilicen en la documentación, los diagramas y el código fuente.

Definición de Bounded Contexts

Para evitar que el modelo se vuelva monolítico e inmanejable, el sistema se dividió en seis **Contextos Delimitados** (*Bounded Contexts*), estableciendo fronteras claras de responsabilidad y límites de consistencia:

1. **Ventas y Caja (Core):** Responsable del flujo de transacción en el punto de venta, emisión de comprobantes, registro de líneas de venta y captura de formas de pago (efectivo, tarjeta, transferencia).
2. **Apartados (Core):** Gestiona el ciclo de vida de las reservas de productos, asegurando el cobro del anticipo mínimo del 25%, el cálculo de la fecha fija de vencimiento a 14 días y la liquidación o cancelación del apartado.
3. **Devoluciones (Core):** Modela el proceso de atención a clientes por garantías o cambios, evaluando políticas de plazo (30 días), la inspección del estado físico del producto y la generación de reembolsos o notas de crédito con supervisión.
4. **Inventario y Seriales (Core):** Administra el stock físico por sucursal, controlando las existencias bajo la fórmula `disponible = total - reservado - dañado - en tránsito`. Asimismo, rastrea la trazabilidad e historial inmutable de cada `NumeroDeSerie`.
5. **Catálogo y Maestros (Supporting):** Provee la información de referencia para productos, categorías, clientes, empleados con sus roles y sucursales físicas.
6. **Auditoría (Generic):** Mantiene una bitácora inmutable de solo lectura (*append-only*) sobre todos los eventos del sistema para asegurar la trazabilidad operacional exigida por la dirección.

Comunicación Inter-Contexto

Para mantener el desacoplamiento, los contextos **nunca comparten referencias directas de objetos o tablas en memoria**. La comunicación entre *Bounded Contexts* se realiza exclusivamente a través de **identificadores primarios** (como `SKU`, `Folio`, `ClienteDocumento`) y mediante la publicación/suscripción de **Eventos de Dominio** (*Domain Events*).

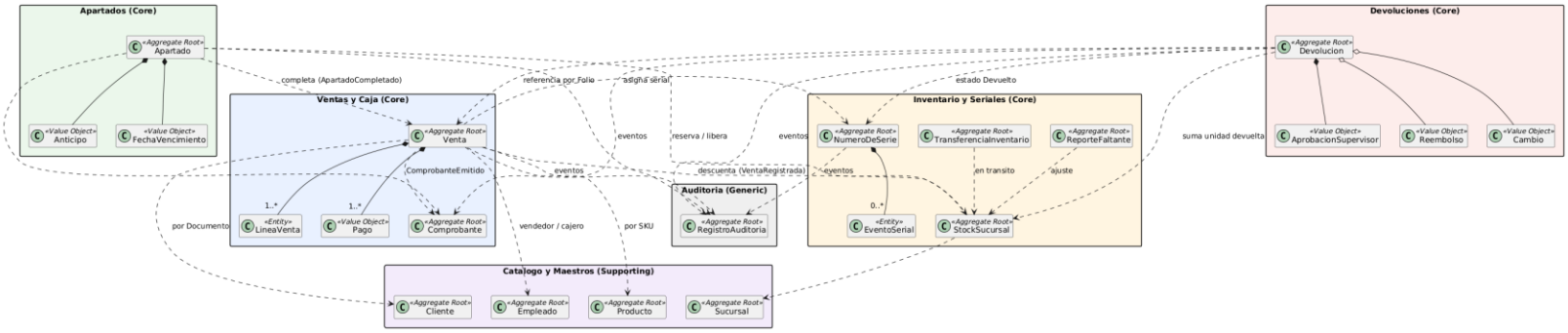


Figura 2. Mapa de Dominio y Bounded Contexts de la arquitectura TecnoStock.

2.1 Lenguaje ubicuo (glosario → código)

Término del glosario	Nombre en el modelo	Tipo DDD
Venta	`Venta`	Aggregate Root
(renglón de la venta)	`LineaVenta` (propuesto)	Entity
Comprobante / Folio	`Comprobante`, `Folio`	Aggregate Root / Value Object
Apartado, Anticipo, Vencimiento de Apartado	`Apartado`, `Anticipo`, `FechaVencimiento`	Aggregate Root / VO / VO
Devolución, Reembolso, Plazo de Devolución	`Devolucion`, `Reembolso`, `PlazoDevolucion`	Aggregate Root / VO / VO
Inventario, Disponible, Reservado	`StockSucursal` (`disponible()`, `reservado`)	Aggregate Root
Número de Serie	`NumeroDeSerie`	Aggregate Root (unicidad global)
Diferencia/Faltante	`ReporteFaltante`	Aggregate Root
(movimiento entre sucursales, US16)	`TransferenciaInventario` (propuesto)	Aggregate Root
Producto	`Producto` (con `PoliticaDevolucion`)	Aggregate Root (catálogo)
Cliente	`Cliente`	Aggregate Root
Vendedor, Cajero, Supervisor, Almacén	`Empleado` con `Rol`	Aggregate Root / VO
Sucursal	`Sucursal`	Aggregate Root
Método de Pago	`MetodoPago` (Efectivo, Tarjeta, Transferencia) y `Pago`	Value Objects
Auditoría	`RegistroAuditoria`	Aggregate (append-only)
Garantía	Fuera del dominio: solo se registra el serial (glosario #24)	—

2.2 Bounded Contexts

Contexto	Responsabilidad	Aggregates	Tipo
Ventas y Caja	Venta, pagos, comprobantes	Venta, Comprobante	Core
Apartados	Reserva con anticipo y vencimiento	Apartado	Core
Devoluciones	Políticas de devolución, reembolso/cambio	Devolucion	Core
Inventario y Seriales	Existencias por sucursal, seriales, transferencias, faltantes	StockSucursal, NumeroDeSerie, TransferenciaInventario, ReporteFaltante	Core
Catálogo y Maestros	Productos, clientes, empleados, sucursales	Producto, Cliente, Empleado, Sucursal	Supporting
Auditoría	Bitácora inmutable de operaciones	RegistroAuditoria	Generic

Los contextos se comunican por **identificadores** (SKU, Folio, ClienteId) y por **Domain Events**, nunca por referencias directas entre Aggregates.

Figura 3. Mapeo de Lenguaje Ubicuo y clasificación de contextos de dominio.

Estructura de Agregados e Invariantes

Cada agregado cuenta con una **Raíz de Agregado** (*Aggregate Root*) que actúa como la puerta de acceso exclusiva para cualquier modificación de su estado interno. Esto garantiza que las invariantes de negocio se cumplan al 100% en todo momento:

- **Agregado Venta:** Garantiza que los productos que requieren número de serie tengan asignado un serial válido y disponible antes de confirmar la transacción. Valida que la suma de los pagos recibidos sea exactamente igual al total calculado.
- **Agregado Apartado:** Protege que el anticipo ingresado sea de al menos el 25% del total del producto y fija de forma inmutable la fecha de vencimiento a 14 días calendarios.
- **Agregado StockSucursal:** Mantiene la integridad de las existencias por sucursal, bloqueando cualquier intento de vender o transferir mercancía asignada al estado de RESERVADO por un apartado activo.
- **Agregado NumeroDeSerie:** Garantiza la unicidad global del serial en todo el sistema, impidiendo que un mismo número de serie aparezca registrado en dos ventas simultáneas o no finalizadas.

2.3 Aggregates, Entidades, Value Objects

Aggregate Root	Entidades internas	Value Objects	Invariantes que protege
Venta	LineaVenta	Folio, Dinero, Pago, MetodoPago, DatosPagoElectronico (últimos 4 dígitos, banco, referencia)	Todo producto con RequiereSerial lleva serial en su línea (100% de seriales registrados); monto pagado = total; vencimiento de devolución = fecha de venta + 30 días
Apartado	—	Anticipo, FechaVencimiento, Dinero	Anticipo ≥ 25% del precio total; vencimiento = creación + 14 días (no configurable); estados Activo → Completado / Cancelado / Vencido

Aggregate Root	Entidades internas	Value Objects	Invariantes que protege
Devolucion	—	PlazoDevolucion, Reembolso, Cambio, AprobacionSupervisor	Requiere comprobante O aprobación de Supervisor; dentro de 30 días y sin daño; reembolso por el método de pago original
StockSucursal	—	Cantidad	disponible = total − reservado − dañado − en tránsito; nunca negativo; un producto RESERVADO no se vende
NumeroDeSerie	EventoSerial (historial)	ValorSerie	Valor único en todo el sistema; un serial no puede estar en dos ventas a la vez; estados Disponible/Vendido/Devuelto/Dañado
Comprobante	—	Folio, TipoComprobante	Folio único y secuencial; tipo Venta/Apartado/Devolución
TransferenciaInventario (propuesto)	—	Cantidad	Cantidad bloqueada "en tránsito" en origen; la autoriza un Supervisor
ReporteFaltante	—	Cantidad	Diferencia = cantidad_sistema − cantidad_física; el ajuste requiere aprobación de Supervisor (faltantes > $1000)

Figura 4. Matriz de Agregados, Entidades, Objetos de Valor e Invariantes del Dominio.

3. Capa de Aplicación (*Service Layer*) y Orquestación de Casos de Uso

Principios de Diseño de la Service Layer

La capa de aplicación implementa el patrón **Service Layer**, definiendo una frontera clara de fachada entre la interfaz de usuario (el PdV Windows) y la lógica interna del dominio. Cada método expuesto por un *Application Service* corresponde exactamente a una intención completa de caso de uso o historia de usuario.

Los *Application Services* se rigen bajo el principio de **coordinación pura sin lógica de negocio**:

1. Reciben una DTO o parámetros primitivos desde la interfaz de usuario.
2. Abren una transacción de base de datos.
3. Recuperan los Agregados necesarios a través de las interfaces de *Repository*.
4. Invocan los métodos de comportamiento del dominio en las raíces de agregado o en los servicios de dominio.
5. Persisten el nuevo estado a través del *Repository*.
6. Publican los eventos de dominio resultantes.
7. Confirman y cierran la transacción.

Catálogo de Servicios de Aplicación

Se diseñaron cuatro servicios de aplicación principales que cubren las 17 Historias de Usuario (US01-US17) identificadas en el caso:

- **VentaAppService:** Expone métodos como ConsultarDisponibilidad(), IniciarVenta(), AgregarProducto(), RegistrarSerial(), PagarEnEfectivo(), PagarConTarjetaOTransferencia(), ConfirmarVenta() e ImprimirComprobante().
- **ApartadoAppService:** Modela operaciones como ConsultarApartadosDeCliente(), CrearApartado(), CompletarApartado() y el proceso batch automático CancelarApartadosVencidos().
- **DevolucionAppService:** Orquesta la atención al cliente con métodos como SolicitarDevolucion(), RegistrarInspeccion(), AutorizarExcepcion() y ResolverDevolucion().
- **InventarioAppService:** Administra las existencias mediante ReportarFaltante(), AprobarAjuste(), BuscarHistorialSerial(), IniciarTransferencia() y ConsultarStockPorSucursal().

3.1 `VentaAppService` (contexto Ventas y Caja)

Método (caso de uso)	US	Actor	Qué orquesta (sin lógica)
`ConsultarDisponibilidad(codigoONombre)`	US01	Vendedor	`ProductoRepository` + consulta de solo lectura a `StockSucursal` de las 5 sucursales; devuelve disponible y apartado por separado
`IniciarVenta(vendedorId, sucursalId, clienteDoc?)`	US02	Vendedor	Crea `Venta` en estado En proceso
`AgregarProducto(ventaId, sku, cantidad)`	US02, US05	Vendedor	Carga `Venta`, `Producto`, `StockSucursal`; el dominio rechaza si el stock está RESERVADO (alerta "Producto reservado para [Cliente]")
`RegistrarSerial(ventaId, sku, valorSerie)`	US03	Vendedor	Carga `NumeroDeSerie`; el dominio valida que no esté vendido; queda bloqueado para otra venta
`PagarEnEfectivo(ventaId, montoRecibido)`	US13	Cajero	Delega en `Venta.registrarPago`; registra en talonario de caja
`PagarConTarjetaOTransferencia(ventaId, metodo, ultimos4, banco, referencia)`	US14	Cajero	Igual, con `DatosPagoElectronico`

Método (caso de uso)	US	Actor	Qué orquesta (sin lógica)
`ConfirmarVenta(ventaId)`	US02	Cajero	`Venta.confirmar()` → `VentaRegistrada`; `StockSucursal.descontar`; `NumeroDeSerie.vender`; `GeneradorFolio`; `ComprobanteRepository.guardar`
`CancelarVenta(ventaId, motivo)`	BPMN ("Cancelar transacción")	Cajero	Libera seriales; registra auditoría
`ImprimirComprobante(folio)`	US12	Cajero	`Comprobante.marcarImpreso()` y puerto de impresión del PdV

3.2 `ApartadoAppService` (contexto Apartados)

Método	US	Actor	Qué orquesta
`ConsultarApartadosDeCliente(documento)`	US15	Vendedor	Lectura; marca los próximos a vencer
`CrearApartado(clienteDoc, sku, cantidad, montoAnticipo, metodoPago)`	US04	Vendedor / Cajero	`Apartado.crear` (el dominio valida ≥ 25% y fija vencimiento = hoy + 14 días); `StockSucursal.reservar`; `Comprobante` tipo Apartado
`CompletarApartado(apartadoId, pago)`	US06	Cliente / Cajero	`Apartado.completar` → `ApartadoCompletado`; crea `Venta` formal con nuevo folio; `StockSucursal` pasa de reservado a descontado; descuenta serial si aplica
`CancelarApartadosVencidos(fechaCorte)`	US07	Sistema (fin de día)	Busca `fechaVencimiento < hoy` y estado Activo; por cada uno: `Apartado.cancelarPorVencimiento`, `StockSucursal.liberar`, notificación al cliente, auditoría. Idempotente (ADR-004)

3.3 `DevolucionAppService` (contexto Devoluciones)

Método	US	Actor	Qué orquesta
`SolicitarDevolucion(folio?, sku, serial?, motivo)`	US08	Cajero / Vendedor	Carga `Venta`; `PoliticaDevolucionService.evaluar` (plazo 30 días, comprobante, > $500); crea `Devolucion`
`RegistrarInspeccion(devolucionId, estado)`	US08, US09	Vendedor / Almacén	`Devolucion.registrarInspeccion` (Aceptable / Dañado)
`AutorizarExcepcion(devolucionId, supervisorId, aprobada, motivo)`	US09	Supervisor	`Devolucion.aplicarAprobacion`; si se rechaza guarda motivo
`RechazarDevolucion(devolucionId, motivo)`	US09	Supervisor	Registra motivo y muestra "No aplica porque [motivo]"
`ResolverDevolucion(devolucionId, tipo, nuevoSku?)`	US08	Cajero	Reembolso por método original, o Cambio con diferencia (pagar, crédito, directo); `StockSucursal.incrementar`; `NumeroDeSerie.registrarDevolucion`; `Comprobante` tipo Devolución

3.4 `InventarioAppService` (contexto Inventario y Seriales)

Método	US	Actor	Qué orquesta
`ReportarFaltante(sucursalId, sku, cantidadFisica, causa)`	US10	Almacén	Crea `ReporteFaltante`; `ConciliadorInventario` calcula diferencia; genera tarea de investigación
`AprobarAjuste(reporteId, supervisorId)`	US10, regla 11	Supervisor	Aplica el ajuste a `StockSucursal`
`BuscarHistorialSerial(valorSerie)`	US11	Almacén / Supervisor	Lectura del historial de `NumeroDeSerie` (venta, cliente, sucursal, vendedor, devolución)
`IniciarTransferencia(sku, cantidad, origenId, destinoId)`	US16	Supervisor	Crea `TransferenciaInventario`; `StockSucursal.bloquearEnTransito` en origen
`RecibirTransferencia(transferenciaId)`	US16	Almacén	Descuenta origen, suma destino
`ConsultarStockPorSucursal()`	US17	Supervisor	Lectura: Producto | Suc1…Suc5 | Total, separando disponible y apartado

Figura 5. Definición de firmas y responsabilidades en la Service Layer.

Separación de Persistencia y Transacciones (*Repositories* y *Unit of Work*)

Para mantener la independencia del dominio frente a la infraestructura, se definieron interfaces de *Repository* expresadas puramente en el lenguaje del negocio, evitando exponer consultas SQL en el modelo:

Repository	Necesidad del dominio	Métodos propuestos	Objeto que recupera o guarda
VentaRepository	Persistir el ciclo de vida de la venta	guardar(), buscarPorFolio()	Agregado Venta
StockRepository	Gestionar existencias y reservas	obtenerStock(), actualizarStock()	Agregado StockSucursal
SerialRepository	Validar y rastrear números de serie	buscarPorValor(), guardarHistorial()	Agregado NumeroDeSerie
ApartadoRepository	Controlar reservas y vencimientos	guardar(), buscarActivosVencidos()	Agregado Apartado

Tabla 2. Interfaces de Repository definidas para el dominio.

Alcance Transaccional (Unit of Work): Cada método de la *Service Layer* opera bajo una unidad de trabajo transaccional. Si un pago falla o un número de serie no está disponible, la transacción realiza un *rollback* completo, garantizando que no existan registros inconsistentes en el inventario o en caja.

Separación de Comandos y Consultas (CQRS Básico)

Se aplicó un esquema de CQRS básico para diferenciar las operaciones que modifican el estado de aquellas que solo extraen información de lectura:

Tipo	Nombre	Datos de entrada	Handler / Servicio	Resultado	Justificación
Command	ConfirmarVenta	ventaId, pagos	VentaAppService	Comprobante y estatus Pagada	Modifica stock, seriales y genera folio financiero
Command	CrearApartado	clienteDoc, sku, anticipo	ApartadoAppService	Apartado activo con vencimiento	Modifica el stock a estado reservado
Query	ConsultarDisponibilidad	codigoONombre	VentaAppService (lectura)	Stock por sucursal (disponible/apartado)	Operación rápida de lectura, no altera estados
Query	ConsultarStockSucursal	sucursalId	InventarioAppService (lectura)	Reporte analítico de inventario	Utilizado para auditoría y supervisión remota

Tabla 3. Comandos y consultas del esquema CQRS básico.

Revisión de la Propuesta con Principios SOLID

Se evaluó el diseño arquitectónico para asegurar un bajo acoplamiento y alta cohesión mediante los siguientes hallazgos concretos:

Principio	Hallazgo en el diseño de TecnoStock
Responsabilidad Única (S)	Los agregados (*Venta*, *Apartado*, *StockSucursal*) concentran únicamente las reglas de su propio estado; la coordinación de casos de uso recae exclusivamente en la *Service Layer*.
Abierto/Cerrado (O)	Las políticas de devolución se ampliaron mediante el uso de especificaciones dinámicas, sin necesidad de modificar el código estructural de la entidad *Devolucion*.
Sustitución de Liskov (L)	Las implementaciones concretas de infraestructura (*SqlVentaRepository*) sustituyen limpiamente a las interfaces del dominio sin alterar el contrato esperado.
Segregación de Interfaces (I)	Las interfaces de repositorio están divididas por agregado específico, evitando que los servicios dependan de métodos o contratos que no utilizan.
Inversión de Dependencias (D)	El núcleo de dominio y la aplicación dependen de abstracciones (interfaces), aislando los detalles tecnológicos de base de datos y de la interfaz del PdV Windows.

Tabla 4. Revisión del diseño con los principios SOLID.

4. Representación Arquitectónica mediante Diagramas UML y ADRs

Diagrama de Clases del Dominio

El diagrama de clases consolida la estructura estática del modelo. Ilustra con precisión las 10 entidades principales, los objetos de valor y las enumeraciones que modelan los estados del sistema. Se representan las dependencias hacia las interfaces del patrón *Repository*, asegurando que la capa de dominio dependa únicamente de abstracciones en cumplimiento con el principio de Inversión de Dependencias (D de SOLID).

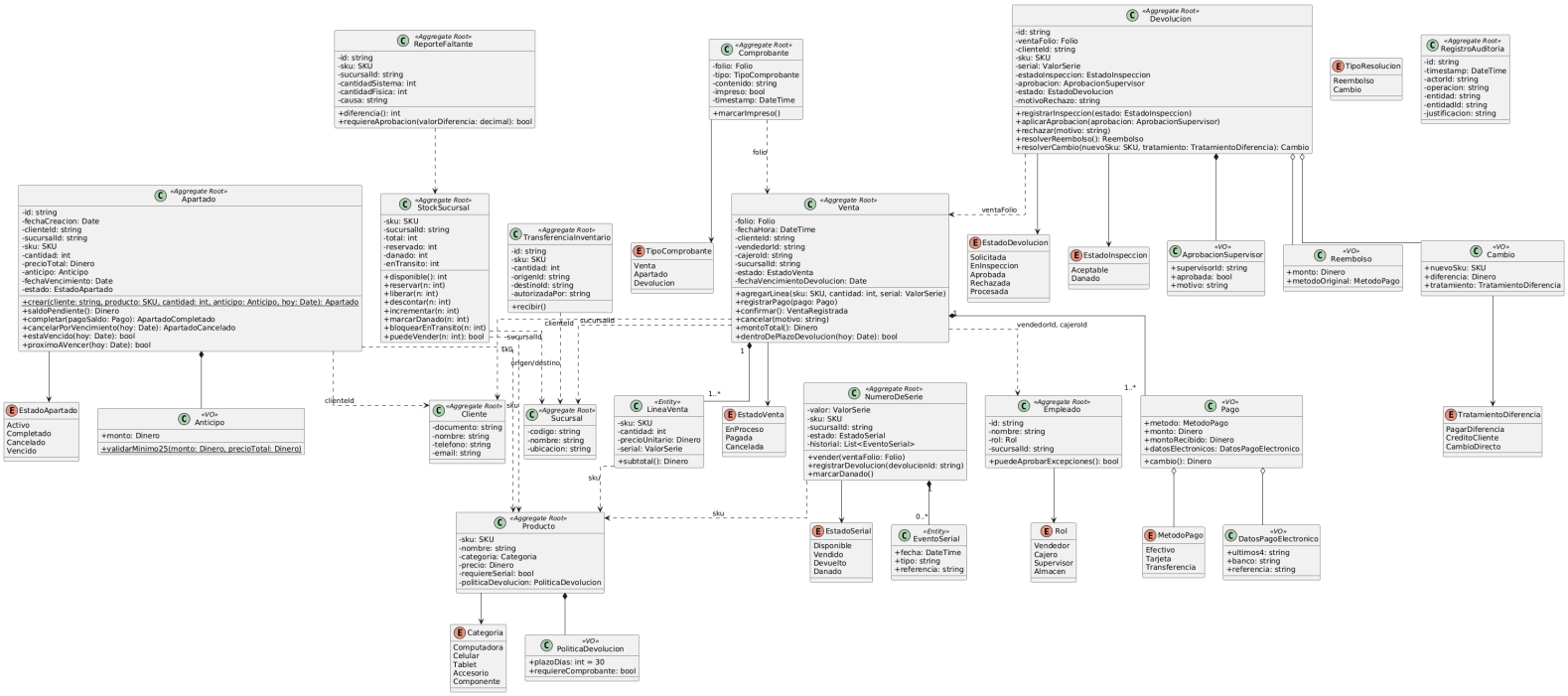


Figura 6. Diagrama de Clases completo de la arquitectura del dominio de TecnoStock.

Diagramas de Secuencia

Se construyeron dos diagramas de secuencia UML para demostrar el comportamiento dinámico del sistema y la interacción transaccional entre las capas durante la ejecución de los casos de uso más complejos:

1. **Diagrama de Secuencia 1 - Venta con Número de Serie y Pago:** Muestra la interacción entre el PdV Windows, el VentaAppService, el agregado StockSucursal (validando existencias y bloqueos de reserva), el agregado NumeroDeSerie (verificando unicidad), la generación del folio secuencial y la emisión de eventos de auditoría tras confirmar el pago.
2. **Diagrama de Secuencia 2 - Devolución con Aprobación de Supervisor:** Modela la bifurcación de la lógica de devolución. Representa la consulta a la política de devolución (30 días), la inspección del estado físico del artículo y la autorización remota por parte del Supervisor cuando se supera el umbral

de $500 o falta el comprobante de pago, concluyendo con el reingreso del producto al inventario y el ajuste del estado del serial.

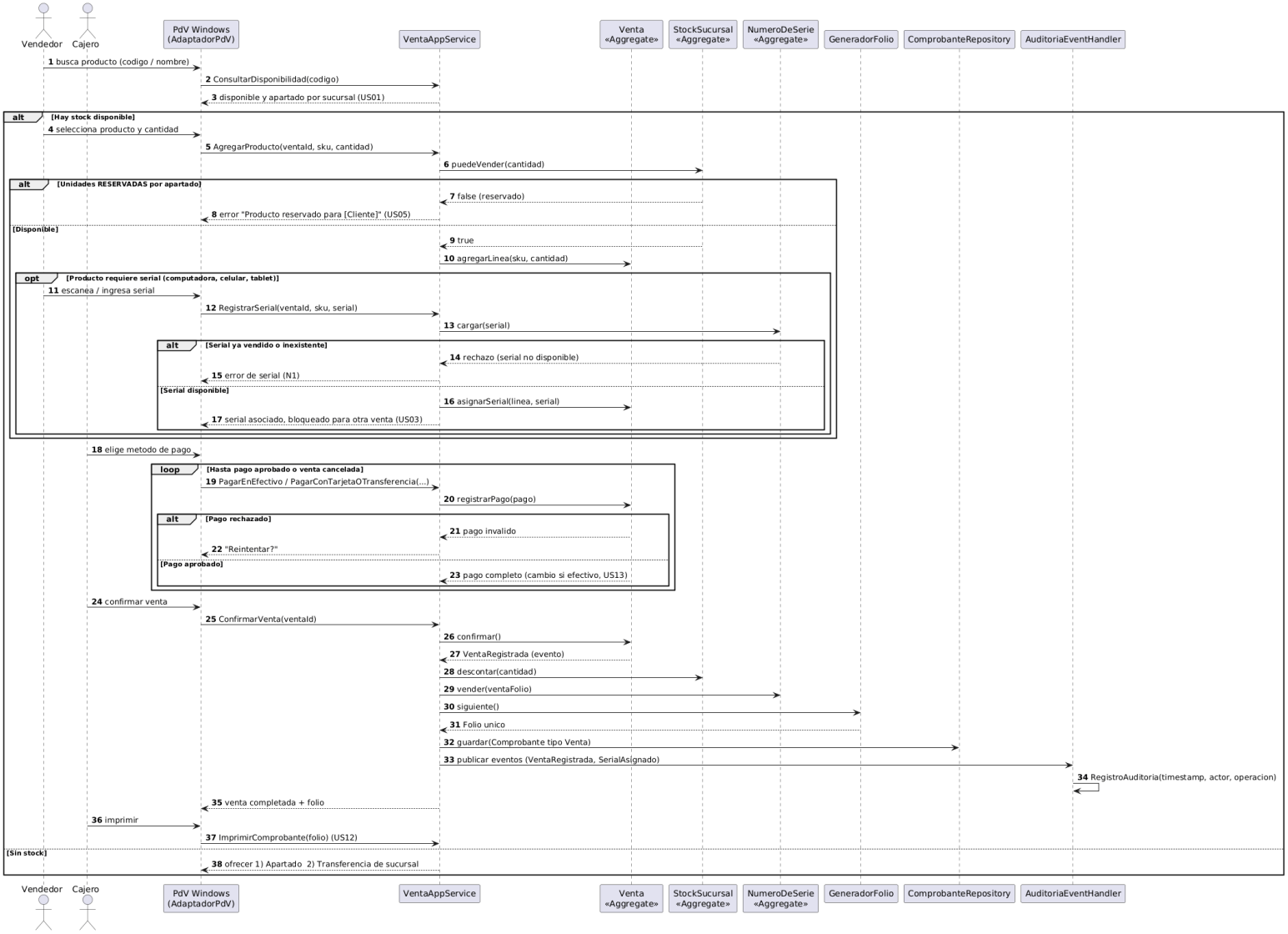


Figura 7. Diagrama de Secuencia UML: Flujo completo de venta con producto serializado y pago.

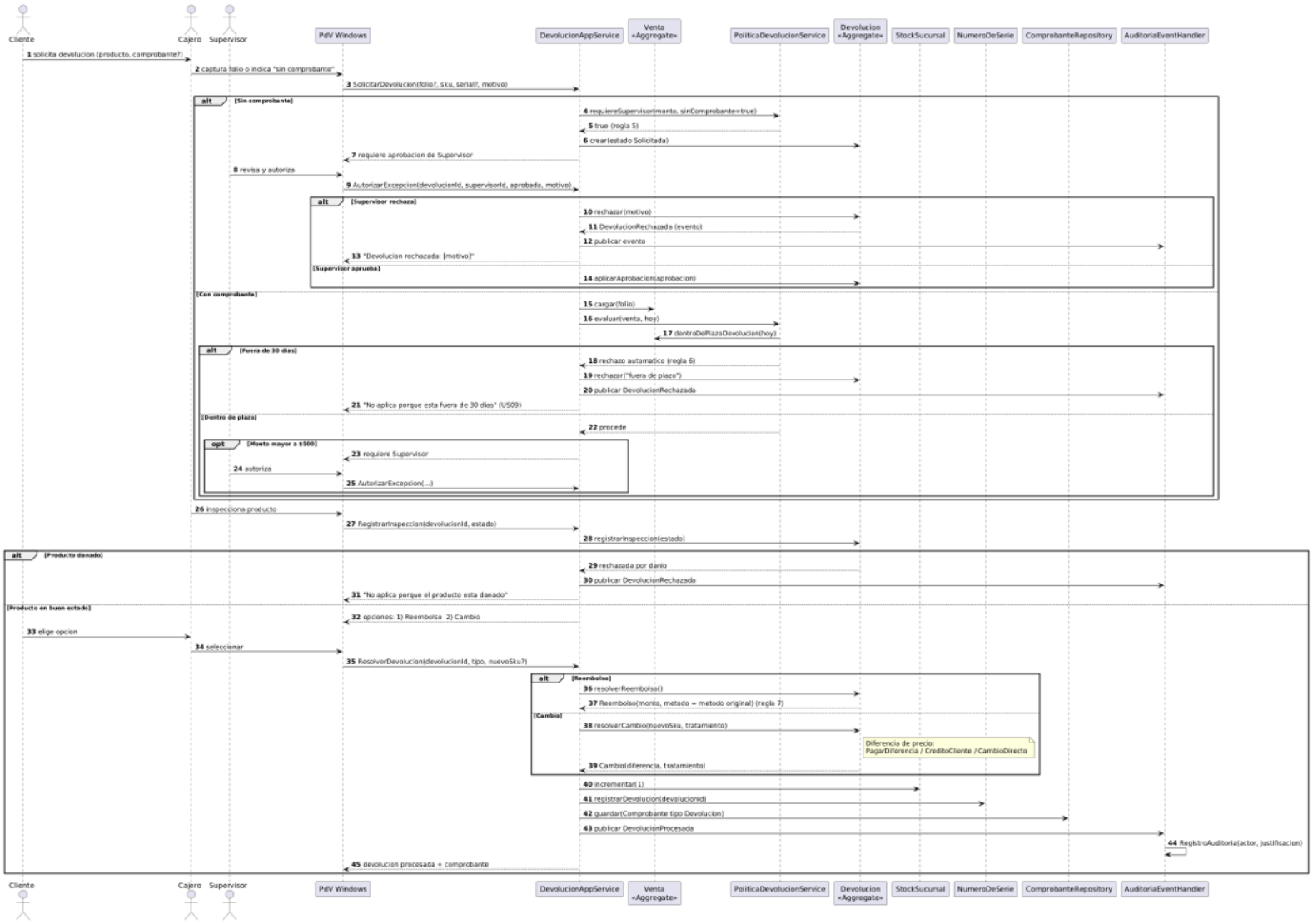


Figura 8. Diagrama de Secuencia UML: Flujo de gestión de devolución con excepciones y aprobación de supervisor.

Registros de Decisiones de Arquitectura (ADR)

Las decisiones más trascendentales del diseño se formalizaron mediante archivos ADR (*Architecture Decision Record*), documentando el estado, contexto, alternativas consideradas, consecuencias positivas/negativas y la trazabilidad con los requerimientos del negocio:

- **ADR-001:** Adopción de *Domain Model* + *Service Layer* como patrón central.
- **ADR-002:** Tratamiento de NumeroDeSerie como un Agregado independiente con unicidad global a nivel sistema.
- **ADR-003:** Mecanismo de reserva de stock con bloqueo optimista de concurrencia para la prevención total de dobles ventas en apartados.
- **ADR-004:** Implementación de un proceso batch diario de fin de día e idempotente para la cancelación de apartados vencidos.

- **ADR-005:** Centralización del motor de reglas de devolución mediante el patrón *Specification* y *Domain Service*.
- **ADR-006:** Implementación de bitácora de auditoría inmutable impulsada por *Domain Events*.
- **ADR-007:** Estructura en capas y patrón *Adapter* para la integración no intrusiva con el sistema PdV Windows actual.

Figura 9. Vista de la documentación de registros ADR estructurados en el repositorio de GitHub.

(Nota borrable: Inserta aquí una captura de tu carpeta docs/adr/ o del archivo 05-adrs.md dentro de GitHub)

5. Bitácora de Uso Responsable de Inteligencia Artificial

Conforme a los lineamientos de la actividad, la Inteligencia Artificial se utilizó estrictamente como un copiloto de apoyo técnico para la consulta conceptual, la estructuración de borradores y la revisión de sintaxis. El equipo conservó en todo momento la autoría y la validación de las decisiones frente al caso TecnoStock:

Iteración / Tema	*Prompt o consulta realizada al asistente*	*Respuesta u orientación obtenida*	*Error, supuesto o ajuste detectado por el equipo*	*Decisión final del equipo*
1. Elección de Patrón	*"¿Cómo puedo justificar técnicamente el uso de Domain Model frente a Transaction Script basándome en las reglas de inventario y devoluciones de una tienda de electrónica?"*	*El asistente explicó los riesgos de la dispersión de lógica en scripts y sugirió estructurar la justificación alrededor de la protección de invariantes y reglas complejas.*	*La sugerencia inicial de la IA mezclaba conceptos de bases de datos relacionales en la capa de lógica.*	*El equipo filtró la explicación, descartó los aspectos de infraestructura y redactó la justificación enfocándose puramente en las reglas de negocio de TecnoStock.*
2. Bounded Contexts y Agregados	*"Ayúdame a revisar si el modelo de inventario y seriales de TecnoStock debe separar el número de serie como un agregado independiente."*	*Propuso modelar el NumeroDeSerie como un Aggregate Root para garantizar unicidad global y rastreabilidad de estados.*	*La propuesta inicial de la IA no contemplaba el almacenamiento de un historial de cambios para las auditorías.*	*Se ajustó el diseño para incorporar una entidad interna de eventos (EventoSerial), validando que cumpliera con las reglas del caso.*
3. Service Layer	*"¿Cómo debo estructurar los métodos del Application Service para que coordinen casos de uso sin*	*Entregó una lista de métodos orientados a intenciones de usuario (como IniciarVenta, CrearApartado,*	*La IA intentó colocar una validación de porcentajes de anticipo dentro del método del servicio.*	*Se rechazó esa ubicación y se reestructuró para que el servicio delegue esa validación*

Iteración / Tema	*Prompt o consulta realizada al asistente*	*Respuesta u orientación obtenida*	*Error, supuesto o ajuste detectado por el equipo*	*Decisión final del equipo*
	absorber reglas de negocio?"	*SolicitarDevolucion) separando la orquestación.*		*estrictamente al interior del agregado de dominio correspondiente.*
4. Repositorios y Unit of Work	*"¿Cuál es la mejor forma de definir interfaces de Repository para que el dominio no dependa de detalles de infraestructura SQL?"*	*Sugirió crear contratos limpios basados en métodos del negocio (como buscarPorFolio u obtenerStock) y explicar el control transaccional por método.*	*La respuesta sugería usar un ORM específico directamente en los contratos de las entidades.*	*Se omitió dicha recomendación y el equipo diseñó las interfaces de repositorio desacopladas utilizando exclusivamente objetos y tipos del dominio.*
5. Principios SOLID	*"¿Cómo puedo verificar si mi diseño cumple con el principio de Inversión de Dependencias (D) en la separación entre dominio e infraestructura?"*	*Explicó que las capas superiores deben apuntar hacia abstracciones (interfaces) y no hacia implementaciones concretas de bases de datos.*	*El ejemplo genérico que dio la IA estaba orientado a frameworks web de Node.js/Python ajenos al proyecto escolar.*	*Se adaptó el concepto teórico para aplicarlo directamente a los contratos de repositorio y los servicios de aplicación de TecnoStock.*
6. Sintaxis UML y PlantUML	*"Muéstrame un ejemplo rápido de sintaxis en PlantUML para representar clases con atributos privados, métodos y enumeraciones."*	*Proporcionó un bloque de código básico de referencia con la estructura de clases y relaciones de agregación.*	*El código de ejemplo venía incompleto y no incluía los nombres específicos de las entidades de TecnoStock.*	*El equipo utilizó el ejemplo únicamente como guía de sintaxis y redactó manualmente todo el diagrama de clases y secuencias adaptado al caso real.*

Tabla 5. Bitácora de iteraciones de uso de IA como copiloto de apoyo.

6. Validación y Trazabilidad de la Propuesta

Se realizó un recorrido exhaustivo sobre los componentes de diseño para asegurar consistencia global:

Aspecto	**Resultado de la validación**
Trazabilidad	Cada Application Service y Agregado se encuentra vinculado directamente con los requerimientos del BRD (N1-N8) y las Historias de Usuario (US01-US17).
Lenguaje	Los términos del glosario se mantuvieron idénticos y unificados en los diagramas UML, el código y los ADR.
Transaccionalidad	Los límites de operación y el manejo de excepciones de negocio quedaron definidos en los servicios y agregados.

Tabla 6. Verificaciones de consistencia y trazabilidad del diseño.

Conclusiones

La elaboración del diseño arquitectónico para la empresa TecnoStock permitió transformar requerimientos de negocio complejos en una solución de software altamente modular, mantenible y alineada con los estándares de la industria. A partir del trabajo realizado, se destacan las siguientes conclusiones clave:

1. **Protección Efectiva de Invariantes Complejas:** El análisis demostró que los problemas principales de TecnoStock no radicaban en la captura o lectura simple de datos, sino en la vulnerabilidad de sus reglas operativas (tales como la duplicación de números de serie, la venta por error de productos apartados y la aplicación desigual de devoluciones). La selección del patrón *Domain Model* garantizó que las validaciones de negocio queden fuertemente protegidas dentro de los propios agregados, eliminando la duplicación de lógica entre pantallas o scripts.
2. **Desacoplamiento Operativo mediante Bounded Contexts:** La división del dominio en seis contextos delimitados bien definidos (*Ventas, Apartados, Devoluciones, Inventario, Catálogo y Auditoría*) permitió establecer límites claros de responsabilidad. Al prohibir referencias directas entre agregados de distintos contextos y utilizar eventos de dominio, el sistema gana la flexibilidad necesaria para escalar o modificar un módulo sin afectar el funcionamiento de los demás.
3. **Integración Transaccional y No Intrusiva con la Service Layer:** La capa de servicios actuó como un puente de orquestación sumamente efectivo. Al centralizar cada caso de uso en métodos transaccionales únicos, se logró una estrategia de integración limpia para el punto de venta (PdV) Windows existente en las sucursales mediante el uso de adaptadores, garantizando la consistencia del sistema sin requerir una reescritura total de la interfaz de usuario.
4. **Trazabilidad y Calidad Documental:** La combinación de diagramas estáticos (clases), dinámicos (secuencia) y registros de decisiones (ADR) proporciona una base técnica transparente para los desarrolladores. La documentación formal de cada ADR asegura que las razones de diseño, los riesgos aceptados y las alternativas descartadas permanezcan visibles a lo largo del ciclo de vida del proyecto.